

Development Roadmap Document

For

Nexora: Multi-Vendor E-Commerce Marketplace

Version: 1.0.0-PROD
Date: July 26, 2026
Prepared By: Office of the Chief Technical Architect & Enterprise Engineering Management Group

1. Cover Page

- **Document Title:** Enterprise Software Development Roadmap
- **Project Name:** Nexora Multi-Vendor E-Commerce Marketplace
- **Version:** 1.0.0-PROD
- **Status:** Approved for Execution

2. Document Information

Attribute	Details
Project Title	Nexora Multi-Vendor E-Commerce Marketplace
Document Type	Software Development Roadmap & Implementation Plan
Target Audience	Executive Engineering Leadership, Core Backend Teams, Frontend Integration Leads, QA Automation Engineers, DevOps Specialists
Security Classification	Proprietary & Confidential
Primary Technology Stack	Java, Spring Boot, Spring MVC, Spring Data JPA, Hibernate, Spring Security, JWT, MySQL, Redis, Apache Kafka, Docker, Swagger, JUnit, Mockito, Maven, React

3. Revision History

Date	Version	Author	Description / Change Summary
July 26, 2026	1.0.0	Chief Technical Architect	Initial production-ready development roadmap fully aligned with Nexora SRS, System Architecture, Database Design, and API Design specifications.

4. Table of Contents

1. [Cover Page](#)
2. [Document Information](#)
3. [Revision History](#)
4. [Table of Contents](#)
5. [Project Vision](#)
6. [Development Philosophy](#)
7. [High-Level Development Phases](#)
8. [Detailed Phase Plan](#)
9. [Learning Alignment](#)
10. [Git Workflow](#)
11. [Coding Standards](#)
12. [Testing Roadmap](#)
13. [Documentation Maintenance](#)
14. [CI/CD Roadmap](#)
15. [Deployment Roadmap](#)
16. [Risk Management](#)
17. [Success Criteria](#)
18. [Future Roadmap](#)
19. [Appendix](#)

5. Project Vision

The **Nexora** engineering initiative establishes a highly scalable, enterprise-grade, production-ready multi-vendor e-commerce platform. Built upon a robust modular monolithic architecture using Java 17 and Spring Boot, Nexora bridges independent third-party merchants with millions of customers. This roadmap outlines the rigorous, incremental engineering path

required to transition Nexora from an empty project repository into a fully tested, containerized, and deployable production system.

6. Development Philosophy

Modular Development

The codebase is structured around distinct domain boundaries (Users, Products, Orders, Payments, Inventory). This isolation prevents tight coupling, simplifies dependency injection, and ensures that modules can be cleanly extracted into independent microservices in future iterations.

Incremental Delivery

Development proceeds in strict, testable phases. The application remains fully deployable and operational at the conclusion of every major milestone, allowing continuous integration and validation.

Clean Code

Codebases adhere strictly to readable, self-documenting naming conventions, small single-responsibility methods, and immutability where applicable. Technical debt is actively eliminated during every sprint.

SOLID Principles

Every class and component strictly follows SOLID design principles, maximizing maintainability, testability, and extensibility across service and repository layers.

Continuous Refactoring

Code is continuously refactored as architectural patterns solidify, ensuring optimal data structures, minimal query overhead, and clean object-oriented design.

Documentation First

API specifications (OpenAPI/Swagger) and database design contracts are established and reviewed prior to backend code implementation.

7. High-Level Development Phases

- **Phase 1:** Project Setup & Environment Initialization
- **Phase 2:** Authentication & Security Foundation
- **Phase 3:** User Management & Profile Services
- **Phase 4:** Seller Management & Storefront Onboarding
- **Phase 5:** Category Taxonomy Management
- **Phase 6:** Product Catalog & Variant Management
- **Phase 7:** Real-Time Inventory & Stock Locking
- **Phase 8:** Customer Shopping Cart Management
- **Phase 9:** Customer Wishlist Services

- **Phase 10:** Master Order Placement & Split Fulfillment
- **Phase 11:** Payment Processing & Transaction Ledgers
- **Phase 12:** Shipment & Courier Logistics Tracking
- **Phase 13:** Customer Reviews & Ratings Engine
- **Phase 14:** Multi-Channel Notification Services
- **Phase 15:** Seller & Administrator Dashboards
- **Phase 16:** Financial Reporting & Audit Logging
- **Phase 17:** Redis Caching & Performance Layer
- **Phase 18:** Apache Kafka Event-Driven Architecture
- **Phase 19:** Comprehensive Testing & QA Automation
- **Phase 20:** Containerization via Docker & Compose
- **Phase 21:** Production Deployment & Monitoring

8. Detailed Phase Plan

Phase 1 – Project Setup

- **Purpose:** Establish foundational repository structure, Maven build configuration, and core properties.
- **Features:** Multi-module Maven setup, Spring Boot parent configuration, MySQL connection properties.
- **Spring Concepts:** Spring Boot Starters, Dependency Management, Application Properties.
- **Classes to Create:** NexoraApplication.java, base configuration classes, global exception handler stubs.
- **Expected Git Commits:** chore: initialize spring boot project, config: add mysql and jpa properties.
- **Testing Strategy:** Context loading test (ApplicationContextTest).
- **Definition of Done:** Project builds successfully via Maven and connects to MySQL database.

Phase 2 – Authentication

- **Purpose:** Implement secure user registration, login, and JWT-based stateless authentication.
- **Features:** Customer/seller registration, login, token refresh, password hashing, RBAC.
- **Spring Concepts:** Spring Security Filters, JWT Generation, Authentication Manager, PasswordEncoder.
- **Classes to Create:** User, Role, UserRepository, AuthService, JwtTokenProvider, JwtAuthenticationFilter, AuthController.
- **Expected Git Commits:** feat: implement user and role entities, feat: add jwt security filter and auth controller.
- **Testing Strategy:** Unit tests for token generation; Mockito tests for auth service.
- **Definition of Done:** Users can register and obtain valid JWT tokens to access protected endpoints.

Phase 3 – User Management

- **Purpose:** Manage customer profile details and physical shipping addresses.
- **Features:** Profile retrieval, profile updates, address CRUD operations, default address selection.
- **Spring Concepts:** REST Controllers, Request Validation, DTO Mapping, Service Layer Transactions.
- **Classes to Create:** CustomerProfile, Address, CustomerRepository, AddressRepository, CustomerService, CustomerController.
- **Expected Git Commits:** feat: add customer profile and address entities, feat: implement customer profile and address controllers.
- **Testing Strategy:** Controller integration tests using MockMvc.
- **Definition of Done:** Customers can manage their profiles and addresses successfully.

Phase 4 – Seller Management

- **Purpose:** Merchant storefront registration and administrative onboarding workflow.
- **Features:** Seller profile creation, business verification, store details, admin approval endpoint.
- **Spring Concepts:** Transaction Management, Role Authorization (hasRole('ADMIN')).
- **Classes to Create:** Seller, SellerRepository, SellerService, SellerController, AdminSellerController.
- **Expected Git Commits:** feat: implement seller entity and onboarding service, feat: add admin seller approval workflow.
- **Testing Strategy:** Service unit tests verifying status transitions.
- **Definition of Done:** Sellers can register and administrators can approve store applications.

Phase 5 – Category Management

- **Purpose:** Establish hierarchical product taxonomy.
- **Features:** Parent-child category creation, slug generation, active status toggling.
- **Spring Concepts:** Self-referential JPA mappings, Repository queries.
- **Classes to Create:** Category, CategoryRepository, CategoryService, CategoryController, AdminCategoryController.
- **Expected Git Commits:** feat: implement hierarchical category entity, feat: add category management endpoints.
- **Testing Strategy:** Repository tests for hierarchical tree traversal.
- **Definition of Done:** Categories and sub-categories can be queried and managed.

Phase 6 – Product Management

- **Purpose:** Enable sellers to create and manage catalog products, variants, and images.
- **Features:** Product CRUD, variant attribute JSON mapping, image asset attachment.
- **Spring Concepts:** One-to-Many cascade persist, JSON column handling, pagination.

- **Classes to Create:** Product, ProductVariant, ProductImage, ProductRepository, ProductService, ProductController, SellerProductController.
- **Expected Git Commits:** feat: implement product and variant entities, feat: add seller product catalog management.
- **Testing Strategy:** Integration tests verifying cascading persistence.
- **Definition of Done:** Sellers can list products with variants and images; customers can browse the catalog.

Phase 7 – Inventory

- **Purpose:** Real-time stock tracking and concurrency control for product variants.
- **Features:** Stock quantity management, low stock threshold alerts, optimistic locking.
- **Spring Concepts:** Optimistic Locking (@Version), Transaction Isolation.
- **Classes to Create:** Inventory, InventoryRepository, InventoryService, InventoryController.
- **Expected Git Commits:** feat: implement inventory stock tracking entity, feat: add optimistic locking for stock updates.
- **Testing Strategy:** Concurrency test simulating simultaneous stock decrements.
- **Definition of Done:** Inventory quantities update safely under concurrent purchase attempts.

Phase 8 – Cart

- **Purpose:** Persistent shopping cart session management for customers.
- **Features:** Cart retrieval, adding items, updating quantities, item removal, cart clearing.
- **Spring Concepts:** Entity associations, Cascade remove, orphan removal.
- **Classes to Create:** Cart, CartItem, CartRepository, CartItemRepository, CartService, CartController.
- **Expected Git Commits:** feat: implement customer shopping cart entities, feat: add cart item management REST endpoints.
- **Testing Strategy:** Unit tests for cart total calculation and item addition.
- **Definition of Done:** Customers can maintain a persistent shopping cart across sessions.

Phase 9 – Wishlist

- **Purpose:** Allow customers to save desired products for future purchase.
- **Features:** Wishlist retrieval, adding products, removing items.
- **Spring Concepts:** Many-to-One and One-to-Many mappings.
- **Classes to Create:** Wishlist, WishlistItem, WishlistRepository, WishlistService, WishlistController.
- **Expected Git Commits:** feat: implement wishlist and wishlist item entities, feat: add wishlist REST controllers.
- **Testing Strategy:** Repository tests for wishlist item association.
- **Definition of Done:** Customers can save and manage favorite products in their wishlist.

Phase 10 – Orders

- **Purpose:** Convert cart items into master purchase orders and split vendor sub-orders.
- **Features:** Checkout validation, order placement, sub-order generation, order cancellation.
- **Spring Concepts:** Complex @Transactional boundaries, Order snapshots.
- **Classes to Create:** Order, SubOrder, OrderItem, OrderRepository, SubOrderRepository, OrderService, OrderController.
- **Expected Git Commits:** feat: implement order and sub-order entities, feat: add checkout and order placement transaction service.
- **Testing Strategy:** Integration test verifying atomic cart-to-order conversion and inventory reservation.
- **Definition of Done:** Customers can place orders, generating isolated sub-orders per vendor.

Phase 11 – Payments

- **Purpose:** Record financial transactions and verify payment gateway callbacks.
- **Features:** Payment intent creation, payment verification, transaction ledger recording, refund processing.
- **Spring Concepts:** Idempotency handling, external API integration stubs.
- **Classes to Create:** Payment, PaymentRepository, PaymentService, PaymentController.
- **Expected Git Commits:** feat: implement payment transaction ledger entity, feat: add payment verification and processing controllers.
- **Testing Strategy:** Unit tests for payment status state machine transitions.
- **Definition of Done:** Payments can be recorded, verified, and linked to master orders.

Phase 12 – Shipping

- **Purpose:** Track courier logistics and milestone updates for sub-orders.
- **Features:** Shipment creation, tracking number assignment, status updates.
- **Spring Concepts:** One-to-One sub-order linkage, lifecycle state management.
- **Classes to Create:** Shipment, ShipmentRepository, ShipmentService, ShipmentController.
- **Expected Git Commits:** feat: implement shipment tracking entity, feat: add seller shipment management endpoints.
- **Testing Strategy:** Integration tests verifying shipment assignment to sub-orders.
- **Definition of Done:** Sellers can assign tracking numbers and update shipment milestones.

Phase 13 – Reviews

- **Purpose:** Capture customer ratings, feedback, and seller replies for purchased products.
- **Features:** Review submission, rating validation, verified purchase checking, seller replies.
- **Spring Concepts:** Business validation rules, composite constraints.
- **Classes to Create:** Review, ReviewRepository, ReviewService, ReviewController.
- **Expected Git Commits:** feat: implement customer product review entity, feat: add

review submission and moderation endpoints.

- **Testing Strategy:** Unit tests enforcing one review per purchased product variant.
- **Definition of Done:** Customers can submit verified reviews and sellers can reply.

Phase 14 – Notifications

- **Purpose:** Deliver multi-channel alerts (Email, In-App) for system and order events.
- **Features:** Notification creation, marking as read, retrieval.
- **Spring Concepts:** Asynchronous execution (@Async), event listeners.
- **Classes to Create:** Notification, NotificationRepository, NotificationService, NotificationController.
- **Expected Git Commits:** feat: implement notification entity and service, feat: add notification retrieval and read status endpoints.
- **Testing Strategy:** Unit tests verifying notification payload creation.
- **Definition of Done:** Users receive notifications for relevant account and order events.

Phase 15 – Dashboard

- **Purpose:** Aggregate metrics and analytics for sellers and administrators.
- **Features:** Sales revenue aggregation, order counts, customer metrics.
- **Spring Concepts:** JPQL aggregation queries, DTO projection.
- **Classes to Create:** DashboardService, SellerDashboardController, AdminDashboardController.
- **Expected Git Commits:** feat: implement seller and admin dashboard metrics services, feat: add analytics dashboard REST endpoints.
- **Testing Strategy:** Repository query tests for sales aggregation calculations.
- **Definition of Done:** Sellers and admins can view aggregated performance metrics.

Phase 16 – Reports

- **Purpose:** Generate financial audit logs and operational reports.
- **Features:** Sales reports, audit log querying, PDF invoice streaming.
- **Spring Concepts:** Response entity streaming, pagination, filtering.
- **Classes to Create:** AuditLog, AuditLogRepository, ReportService, AdminReportController, OrderInvoiceController.
- **Expected Git Commits:** feat: implement audit log entity and security logging aspect, feat: add administrative reporting and invoice download endpoints.
- **Testing Strategy:** Integration tests for invoice generation and audit log persistence.
- **Definition of Done:** Administrators can generate reports and download PDF invoices.

Phase 17 – Redis

- **Purpose:** Implement distributed caching for catalog searches and rate limiting.
- **Features:** Product caching, session token blocklisting, IP rate limiting.
- **Spring Concepts:** Spring Data Redis, RedisTemplate, Cache Abstraction.
- **Classes to Create:** RedisConfig, CacheService, rate-limiting filter.

- **Expected Git Commits:** feat: configure spring data redis and cache manager, feat: implement redis rate limiting and catalog caching.
- **Testing Strategy:** Integration tests with embedded Redis or testcontainers.
- **Definition of Done:** Frequent catalog queries are cached and rate limits are enforced.

Phase 18 – Kafka

- **Purpose:** Implement asynchronous event-driven messaging for decoupled services.
- **Features:** Order placement event publishing, notification event consumption.
- **Spring Concepts:** Spring Kafka, ProducerFactory, ConcurrentKafkaListenerContainerFactory.
- **Classes to Create:** KafkaProducerConfig, KafkaConsumerConfig, event publisher/consumer components.
- **Expected Git Commits:** feat: configure apache kafka producer and consumer, feat: implement order event publishing and asynchronous notification handling.
- **Testing Strategy:** Embedded Kafka unit tests for message production and consumption.
- **Definition of Done:** Order events trigger asynchronous notifications via Kafka.

Phase 19 – Testing

- **Purpose:** Ensure comprehensive test coverage across unit, integration, and security layers.
- **Features:** JUnit 5 test suites, Mockito mocks, MockMvc controller tests, Testcontainers.
- **Spring Concepts:** @SpringBootTest, @WebMvcTest, @DataJpaTest.
- **Classes to Create:** Comprehensive test classes across all domain packages.
- **Expected Git Commits:** test: add comprehensive unit and integration test suites, test: achieve minimum 80 percent code coverage.
- **Testing Strategy:** Automated test execution via Maven build pipeline.
- **Definition of Done:** All test suites pass successfully with required code coverage.

Phase 20 – Docker

- **Purpose:** Containerize backend application and supporting infrastructure.
- **Features:** Multi-stage Dockerfile, Docker Compose orchestration for MySQL, Redis, Kafka, and Spring Boot.
- **Spring Concepts:** Externalized configuration via environment variables.
- **Classes to Create:** Dockerfile, docker-compose.yml.
- **Expected Git Commits:** chore: add multi-stage dockerfile for spring boot backend, chore: add docker-compose orchestration for mysql redis and kafka.
- **Testing Strategy:** Container smoke tests verifying inter-service connectivity.
- **Definition of Done:** Entire application stack boots successfully via Docker Compose.

Phase 21 – Deployment

- **Purpose:** Prepare production deployment configurations and monitoring probes.
- **Features:** Health checks, readiness probes, production logging profiles.

- **Spring Concepts:** Spring Boot Actuator, Micrometer.
- **Classes to Create:** Production application properties, actuator configurations.
- **Expected Git Commits:** chore: configure spring boot actuator and production logging, docs: finalize deployment operational runbook.
- **Testing Strategy:** End-to-end smoke testing against deployed environment.
- **Definition of Done:** Nexora backend is fully deployed, monitored, and operational in production.

9. Learning Alignment

Milestone / Phase	Spring Core	Spring Boot	Spring MVC	JPA / Hibernate	Security / JWT	Redis / Kafka	Testing / DevOps
Phase 1-2	IoC / DI	Starters	REST Controllers	Basic Entities	Security Filter	-	JUnit Setup
Phase 3-6	Bean Scopes	Auto-config	Validation	Associations	RBAC Annotations	-	Mockito Unit Tests
Phase 7-12	Transactions	Profiles	Exception Mapping	Locking / Cascade	Method Security	-	MockMvc Testing
Phase 13-18	Async Execution	Actuator	Streaming	Projections	Rate Limiting	Redis / Kafka	Testcontainers
Phase 19-21	Configuration	External Config	OpenAPI / Swagger	Schema Validation	Production Hardening	Event Streaming	Docker / Compose

10. Git Workflow

Repository Structure

Standard single-repository (monorepo) structure separating backend source code, configuration, database migration scripts, and documentation assets.

Branch Strategy (GitFlow)

- **main:** Production-ready code. Only receives merged pull requests from release/* or verified hotfixes.
- **develop:** Integration branch for ongoing feature development.
- **feature/*:** Feature branches branched from develop for specific roadmap phases.
- **release/*:** Release staging branches for final QA validation before merging to main.
- **hotfix/*:** Emergency production patch branches.

Commit Message Convention (Conventional Commits)

- **feat:** add customer registration endpoint
- **fix:** resolve optimistic locking concurrency exception in inventory
- **refactor:** optimize product category repository query performance
- **test:** add integration test suite for order placement
- **chore:** update docker-compose service definitions

Pull Request Guidelines

- Mandatory peer review by at least one Senior Backend Engineer.
- All CI/CD pipeline checks and automated tests must pass successfully.
- No merge conflicts with the target integration branch.

Release Tags

Semantic versioning tags applied upon merging to main (e.g., v1.0.0-PROD).

11. Coding Standards

- **Package Naming:** Lowercase dot-separated nomenclature (com.nexora.backend.module.domain).
- **Class Naming:** PascalCase nouns and noun phrases (ProductService, OrderController).
- **Method Naming:** camelCase verb phrases (calculateOrderTotal, findByEmail).
- **DTO Naming:** PascalCase suffixed with Request or Response (ProductCreateRequest, UserResponse).
- **Controller Naming:** PascalCase suffixed with Controller (AuthController, ProductController).
- **Repository Naming:** PascalCase suffixed with Repository (UserRepository, OrderRepository).
- **Entity Naming:** Singular PascalCase nouns matching database tables (User, ProductVariant).
- **Service Naming:** PascalCase suffixed with Service (UserService, PaymentService).

12. Testing Roadmap

- **Unit Testing:** Isolated class testing using JUnit 5 and Mockito for service business logic.

- **Integration Testing:** Spring Boot context testing using @SpringBootTest and H2/Testcontainers.
- **Repository Testing:** Data layer validation using @DataJpaTest against MySQL test instances.
- **Controller Testing:** REST API contract and validation testing using MockMvc.
- **Security Testing:** Authentication and authorization filter chain validation testing.
- **API Testing:** End-to-end integration testing using automated Postman/Newman or REST Assured test suites.

13. Documentation Maintenance

- **README:** Updated at the completion of Phase 1 and Phase 20 with setup and execution instructions.
- **API Docs:** Automatically generated and updated via OpenAPI/Swagger annotations during controller development.
- **Architecture:** Maintained and verified against system design documents during phase transitions.
- **Database Design:** Synchronized with JPA entity mappings and migration scripts.
- **Release Notes:** Compiled at each major milestone and production release tag.

14. CICD Roadmap

- **GitHub Actions:** Automated CI/CD pipeline triggered on pull requests and merges.
- **Build:** Automated Maven clean install compilation.
- **Test:** Automated execution of unit and integration test suites.
- **Docker Image:** Automated multi-stage Docker build and container registry push.
- **Deployment:** Automated container orchestration deployment to staging and production environments.

15. Deployment Roadmap

- **Development Environment:** Local developer workstation running via Maven and Docker Compose for supporting databases.
- **Testing Environment:** Staging cluster running containerized services for automated QA and integration testing.
- **Production Environment:** Cloud-hosted container orchestrator (AWS ECS / Kubernetes) with managed MySQL, Redis, and Kafka clusters.
- **Docker Compose:** Standardized orchestration definition for local and staging deployments.
- **Cloud Readiness:** 12-factor app compliance with externalized configuration, stateless processes, and health probes.

16. Risk Management

Risk Identified	Impact	Probability	Mitigation Strategy

High Concurrency Inventory Over-Allocation	High	Medium	Implement JPA optimistic locking (@Version) and Redis distributed locks.
Database Migration Schema Conflicts	High	Low	Enforce strict version-controlled schema migrations and code review gates.
JWT Token Compromise or Hijacking	High	Medium	Enforce short-lived access tokens (15 mins), secure refresh tokens, and Redis revocation lists.
CI/CD Pipeline Build Failures	Medium	Medium	Maintain rigorous local testing standards and containerized test execution environments.

17. Success Criteria

1. 100% completion of all Version 1 features defined in SRS, System Architecture, Database Design, and API Design documents.
2. Successful execution of all unit, integration, and security test suites with minimum 80% code coverage.
3. Successful containerized deployment of the entire backend stack via Docker Compose.
4. Complete OpenAPI/Swagger documentation coverage across all REST endpoints.
5. Zero critical security vulnerabilities identified during static code analysis and audit reviews.

18. Future Roadmap

Version 2.0 Goals

1. Real-time customer support chat microservice.

2. Automated product return and refund workflow management.
3. Customer loyalty reward points and tier management system.
4. AI-powered personalized product recommendation engine.
5. Elasticsearch integration for advanced full-text and semantic search.
6. WebSocket-based live order tracking and inventory streaming.
7. Multi-currency exchange rate conversion engine.
8. B2B bulk quotation negotiation modules.
9. Augmented reality (AR) 3D product model asset hosting.
10. Automated multi-warehouse shipping route optimization.

Version 3.0 Goals

1. Full extraction of modular monolith domains into independent microservices.
2. Event-sourcing architecture for immutable financial ledgers.
3. Multi-region active-active database replication topology.
4. Enterprise multi-tenant tenant isolation layer.
5. Advanced predictive demand forecasting for merchants.

19. Appendix

Appendix A: Reference Standards

- Software Requirements Specification (SRS) for Nexora.
- System Architecture Document for Nexora.
- Database Design Document (DDD) for Nexora.
- REST API Design Specification for Nexora.